Masinsko ucenje

Zadatak 1: Teorijska postavka i izvodjenje sistema najmanjih kvadrata

Opis zadatka: Razmatra se polinomijalni regresioni model y(x, w) = sum_{j=0}^M w_j * x^j sa ciljem aproksimacije zavisnosti izmedju ulazne promenljive X i izlazne Y.
1. Formulisati funkciju cilja Err(w) = (1 / 2N) * sum_{i=1}^N (y_i - y(x_i, w))^2 i razjasniti njeno znacenje.
2. Odrediti parcijalne izvode dErr/dw_k = 0 i izvesti sistem jednadzbi.
3. Predstaviti sistem u matricnom obliku (Phi^T * Phi) * w = Phi^T * y.
4. Objasniti kako se izracunava vektor koeficijenata w.
5. Prodiskutovati uslove egzistencije i jedinstvenosti resenja na osnovu ranga matrice Phi.

Postupak i resenje:
• **1. Kriterijum najmanjih kvadrata:** Funkcija greske meri prosecno polovinsko kvadratno odstupanje izmerenih vrednosti y_i od vrednosti koje daje model. Kvadratna forma penalizuje veca odstupanja sto osigurava da model tezi sto manjem ukupnom odstupanju.
• **2. Parcijalni izvodi:** Izracunavanjem izvoda po w_k dobijamo dErr/dw_k = -(1/N) * sum_{i=1}^N (y_i - y(x_i, w)) * x_i^k. Postavljanjem izvoda na 0 dobija se sistem od M+1 linearne jednacine: sum_{j=0}^M w_j * (sum_{i=1}^N x_i^{j+k}) = sum_{i=1}^N y_i * x_i^k za k = 0, ..., M.
• **3. Matricni zapisi:** Definisanjem Vandermonde matrice dizajna Phi dimenzija N x (M+1) gde je Phi_{ij} = x_i^j, dobijeni sistem prelazi u matricnu formu (Phi^T * Phi) * w = Phi^T * y.
• **4. Izracunavanje parametara:** Vektor w predstavlja resenje sistema. Kada je matrica Phi^T * Phi ne-singularna, w se dobija direktno kao w = (Phi^T * Phi)^{-1} * Phi^T * y. U praksi se resenje stabilno racuna preko numpy.linalg.lstsq ili solve.
• **5. Uslovi za resenje:** Jedinstveno resenje postoji ako i samo ako je matrica Phi punog ranga po kolonama, odnosno rang(Phi) = M + 1 (za sta je potrebno N >= M + 1 i razliciti ulazi x_i). Ako je rang(Phi) < M + 1, matrica je singularna i postoji beskonacno mnogo resenja.

Zadatak 2: Implementacija regresione klase (poly_regression.py)

Opis zadatka: Kreirati klasu PolynomialRegression u datoteci poly_regression.py uskladjenu sa scikit-learn standardom (nasledjuje BaseEstimator i RegressorMixin). U metodi fit izracunati w preko sistema (Phi^T * Phi) * w = Phi^T * y, u predict vracati procene, a u score racunati MSE.

Izvorni Python kod:

```
import numpy as np
import pandas as pd
from sklearn.base import BaseEstimator, RegressorMixin

class PolynomialRegression(BaseEstimator, RegressorMixin):
    """
    Sopstveni regresioni model za polinomijalnu aproksimaciju sa L2 kaznenim clanom.
    Kompatibilan sa scikit-learn standardom.
    """
    def __init__(self, degree=1, alpha=0.0):
        self.degree = degree
        self.alpha = alpha
        self.coef_ = None

    def _build_design_matrix(self, input_vector):
        x_flat = np.asarray(input_vector).ravel()
        # Formiranje Vandermonde matrice reda N x (degree + 1)
        return np.vander(x_flat, N=self.degree + 1, increasing=True)

    def fit(self, X, y):
        design_phi = self._build_design_matrix(X)
        n_obs, n_params = design_phi.shape

        # Izracunavanje matrice sistema i uvodjenje regularizacije
        phi_t = design_phi.T
        matrix_a = phi_t @ design_phi

        # Matrix regularizacije L2 (bez uticaja na slobodni clan w0)
        reg_matrix = np.eye(n_params)
        reg_matrix[0, 0] = 0.0

        system_lhs = matrix_a + self.alpha * reg_matrix
        system_rhs = phi_t @ y

        # Analiticko nalazenje parametara w
        self.coef_ = np.linalg.lstsq(system_lhs, system_rhs, rcond=None)[0]
        return self
```

```
    def predict(self, X):
        mat_x = self._build_design_matrix(X)
        return np.matmul(mat_x, self.coef_)

    def score(self, X, y):
        preds = self.predict(X)
        errors = y - preds
        return float(np.mean(errors ** 2))

    def err_function(self, X, y):
        return 0.5 * self.score(X, y)
```

Zadatak 3: Evaluacija modela razlicitih stepena M (zadatak3_analiza.py)

Opis zadatka: Obuciti modele za stepene M in {1, 2, 3, 5, 9} na podacima iz Tabele 1, izracunati greske obucavanja (Err i MSE) i predvideti biomasu na novim tackama X in {600, 900, 1300, 1600} mm. Analizirati podprilagodjavanje i preprilagodjavanje.

Izvorni Python kod:

```
import numpy as np
import pandas as pd
from poly_regression import PolynomialRegression

def evaluate_models():
    # Definisani podaci iz Tabele 1
    raw_data = {
        'Padavine_mm': [520, 610, 700, 820, 910, 1030, 1120, 1250, 1390, 1510],
        'Biomasa_tha': [3.1, 3.8, 4.0, 5.1, 5.3, 6.0, 6.2, 5.9, 5.1, 4.1]
    }
    dataframe = pd.DataFrame(raw_data)
    x_vals = dataframe['Padavine_mm'].values
    y_vals = dataframe['Biomasa_tha'].values

    # Testne tacke za proveru ekstrapolacije
    x_test = np.array([600, 900, 1300, 1600])

    print("=" * 70)
    print("ZADATAK 3: ANALIZA UTICAJA STEPENA POLINOMA M NA KVALITET MODELA")
    print("=" * 70)

    degrees_list = [1, 2, 3, 5, 9]

    for degree in degrees_list:
        estimator = PolynomialRegression(degree=degree, alpha=0.0)
        estimator.fit(x_vals, y_vals)

        train_mse = estimator.score(x_vals, y_vals)
        train_err = estimator.err_function(x_vals, y_vals)
        test_preds = estimator.predict(x_test)

        print(f"\n[Stepen M = {degree}]")
        print("Procenjeni koeficijenti w:", [round(val, 6) for val in estimator.coef_])
        print(f"Funkcija greske Err(w): {train_err:.6f}")
        print(f"Srednja kvadratna greska (MSE): {train_mse:.6f}")
        print("Predikcije na novim tackama:")
        for pt, pred in zip(x_test, test_preds):
            print(f"  X = {pt:4d} mm --> Predvidjena biomasa = {pred:.2f} t/ha")

if __name__ == "__main__":
    evaluate_models()
```

Rezultati i analiza ponasanja modela:

• **M = 1 (Underfitting):** Model ostvaruje visoku gresku MSE = 0.7485. Obicna prava linija ne moze da opise opadajuci trend biomase pri vecim padavinama.

• **M = 2 i M = 3 (Optimalni opseg):** Znatno smanjuju gresku (MSE = 0.0576 za M=2, MSE = 0.0166 za M=3) i daju fizicki realne predikcije pri ekstrapolaciji.

• **M = 5:** Daje gresku MSE = 0.0112 uz stabilan oblik funkcije.

• **M = 9 (Overfitting):** Postize savrsenu gresku na obuci MSE = 0.0000 jer prolazi kroz sve tacke, ali van dometa obuke izuzetno oscilira i daje potpuno nerealnu predikciju od 77.19 t/ha za X = 1600 mm.

Zadatak 4: Analiza L2 (Ridge) regularizacije (zadatak4_regularizacija.py)

Opis zadatka: Prosiriti model uvodjenjem L2 regularizacije sa parametrom alpha kako bi se sprecio prekomerni rast koeficijenata. Formulisati korigovani sistem i uporediti efekte.

Formulacija i Python kod:
Optimizacioni problem: Err_reg(w) = (1 / 2N) * sum_{i=1}^N (y_i - y(x_i, w))^2 + (alpha / 2) * sum_{j=1}^M w_j^2.
Korigovani matricni sistem: (Phi^T * Phi + alpha * I) * w = Phi^T * y.

```
import numpy as np
import pandas as pd
from poly_regression import PolynomialRegression

def test_l2_regularization():
    raw_data = {
        'X': [520, 610, 700, 820, 910, 1030, 1120, 1250, 1390, 1510],
        'Y': [3.1, 3.8, 4.0, 5.1, 5.3, 6.0, 6.2, 5.9, 5.1, 4.1]
    }
    df = pd.DataFrame(raw_data)
    X_train, y_train = df['X'].values, df['Y'].values
    eval_points = np.array([600, 900, 1300, 1600])

    print("=" * 70)
    print("ZADATAK 4: PRIMENA L2 REGULARIZACIJE (RIDGE REGRESIJA)")
    print("=" * 70)

    for m in [3, 5, 9]:
        print(f"\n>>> Ispitivanje modela za M = {m} <<<")
        for alpha_param in [0.0, 1e-3, 1.0, 100.0]:
            ridge_fit = PolynomialRegression(degree=m, alpha=alpha_param)
            ridge_fit.fit(X_train, y_train)

            mse = ridge_fit.score(X_train, y_train)
            predictions = ridge_fit.predict(eval_points)
            w_norm = np.linalg.norm(ridge_fit.coef_)

            print(f"Alpha: {alpha_param:<7} | MSE: {mse:.6f} | ||w||: {w_norm:.3e} | Predikcije: {np.round(pred
ictions, 2)}")

if __name__ == "__main__":
    test_l2_regularization()
```

Zakljucak: Uvodjenjem faktora alpha > 0 penalizuju se visoke vrednosti parametara w. Kod M = 9, uvodjenjem blage regularizacije (alpha = 0.001) norma koeficijenata ||w|| se drasticno redukuje, cime se eliminise pojava overfittinga i dobijaju stabilne i pouzdane predikcije.